Amazon Aurora PostgreSQL

Complete DBA Handbook

Step-by-step procedures, runbooks and solutions
for day-to-day Aurora PostgreSQL administration

Edition: October 2026
Always validate commands in a non-production cluster first.

Table of Contents

1. About This Handbook 6

1.1 Who this is for . . . 6

1.2 Prerequisites and tooling . . . 6

1.3 Conventions used . . . 6

2. Aurora PostgreSQL Architecture for DBAs 7

2.1 Cluster, instances and shared storage . . . 7

2.2 Endpoints . . . 7

2.3 What is different from community PostgreSQL . . . 7

3. Daily, Weekly and Monthly DBA Checklists 8

3.1 Daily checks . . . 8

3.2 Weekly checks . . . 8

3.3 Monthly and quarterly checks . . . 8

4. Connecting and Basic Access 9

4.1 Connect with psql . . . 9

4.2 Identify where you are connected . . . 9

4.3 Essential psql commands . . . 9

4.4 Cannot connect: step-by-step diagnosis . . . 9

5. Provisioning a Cluster (Step by Step) 11

5.1 Pre-requisites . . . 11

5.2 Step 1: create parameter groups . . . 11

5.3 Step 2: choose the engine version . . . 11

5.4 Step 3: create the cluster . . . 11

5.5 Step 4: add the writer and readers . . . 12

5.6 Step 5: verify . . . 12

5.7 Step 6: post-creation baseline . . . 12

5.8 Choosing instance classes . . . 12

6. Parameter Management 13

6.1 Two levels of parameter groups . . . 13

6.2 Change a parameter (procedure) . . . 13

6.3 Recommended baseline parameters . . . 14

6.4 Connections and memory . . . 14

7. Monitoring and Alerting 15

7.1 Key CloudWatch metrics . . . 15

7.2 Create an alarm . . . 15

7.3 Enhanced Monitoring and Performance Insights . . . 15

7.4 Logs . . . 16
7.5 In-database monitoring queries (psql) . . . 16
7.6 pg_stat_statements: find your most expensive SQL . . . 17
7.7 Event notifications . . . 17

8. Vacuum, Bloat and Transaction ID Wraparound 18

8.1 Daily health queries . . . 18
8.2 What blocks vacuum from cleaning (check all four) . . . 18
8.3 Tune autovacuum . . . 18
8.4 Manual vacuum and analyze . . . 19
8.5 Measure and fix bloat . . . 19
8.6 Emergency: wraparound protection . . . 19

9. Performance Tuning Workflow 20

9.1 Method . . . 20
9.2 Reading EXPLAIN . . . 20
9.3 Index management . . . 20
9.4 Statistics . . . 21
9.5 Connection management . . . 21
9.6 Plan stability . . . 21
9.7 Scale reads and use the cache wisely . . . 21

10. Locks, Blocking and Deadlocks 22

10.1 Find who is blocking whom . . . 22
10.2 Resolve a blocking incident . . . 22
10.3 Safe DDL practice on busy tables . . . 22
10.4 Deadlocks . . . 22

11. Backup, Restore and Cloning 23

11.1 How Aurora backup works . . . 23
11.2 Check backup status . . . 23
11.3 Procedure A: manual snapshot (before every risky change) . . . 23
11.4 Procedure B: restore from a snapshot . . . 23
11.5 Procedure C: point-in-time restore (PITR) . . . 24
11.6 Procedure D: clone for testing . . . 24
11.7 Procedure E: recover from an accidental DROP/DELETE . . . 24
11.8 Other backup options . . . 24
11.9 Backup verification schedule . . . 25

12. High Availability, Failover and Scaling 26

12.1 How failover works . . . 26
12.2 Planned failover (test or maintenance) . . . 26
12.3 Add, remove and tune readers . . . 26
12.4 Vertical scaling (change instance class) . . . 26

12.5 Reader Auto Scaling . . . 27
12.6 Cluster cache management . . . 27
12.7 Disaster recovery: Aurora Global Database . . . 27

13. Upgrades and Maintenance 28

13.1 Maintenance windows and pending actions . . . 28
13.2 Minor version upgrade (procedure) . . . 28
13.3 Major version upgrade (procedure) . . . 28
13.4 Lower-downtime alternatives . . . 29
13.5 Extension and certificate hygiene . . . 29

14. Security Administration 30

14.1 Role model (least privilege) . . . 30
14.2 Authentication . . . 30
14.3 Network and encryption . . . 31
14.4 Auditing with pgaudit . . . 31
14.5 Access review queries . . . 31
14.6 Security checklist . . . 32

15. Replication, Logical Replication and Migration 33

15.1 Aurora replicas versus logical replication . . . 33
15.2 Set up logical replication (publisher in Aurora) . . . 33
15.3 Migrating a database to Aurora PostgreSQL (plan) . . . 33

16. Storage and Temporary Space 35

16.1 Two kinds of storage . . . 35
16.2 Investigate cluster volume growth . . . 35
16.3 Investigate local storage (temp) pressure . . . 35

17. Troubleshooting Runbooks 36

R1. High CPU . . . 36
R2. Too many connections / connection exhaustion . . . 36
R3. Replica lag and reader query cancellations . . . 37
R4. Slow query or sudden plan regression . . . 37
R5. Idle-in-transaction and long-running transactions . . . 37
R6. Lock pile-up (application hangs) . . . 37
R7. Low local storage or disk full . . . 38
R8. Transaction ID wraparound risk . . . 38
R9. Unexpected restart or failover . . . 38
R10. Cannot connect to the cluster . . . 39
R11. Logical replication slot lag or log growth . . . 39
R12. High memory use or swapping . . . 39

18. Extensions and Useful Tools 40

18.1 Installing an extension 40
18.2 Scheduling jobs with pg_cron 40
18.3 Declarative partitioning basics 40
18.4 Import/export with Amazon S3 41

19. Cost, Change Management and Operations Governance **42**

19.1 Cost controls 42
19.2 Change management checklist (any production change) 42
19.3 Incident record template 42

Appendix A. AWS CLI Quick Reference **43**

Appendix B. SQL Quick Reference **44**

Appendix C. References **45**

1. About This Handbook

This handbook is a practical, step-by-step guide for people who administer **Amazon Aurora PostgreSQL-Compatible Edition** in production. Every chapter is written as a procedure: what to check, the exact command or query to run, what a healthy result looks like, and what to do when it is not.

1.1 Who this is for

- Database administrators moving from on-premises PostgreSQL or Oracle to Aurora PostgreSQL.
- Cloud and DevOps engineers who own an Aurora cluster and need a runbook.
- On-call engineers who need a fast diagnosis path (see Chapter 17, Troubleshooting Runbooks).

1.2 Prerequisites and tooling

Tool	Purpose	Notes
psql (PostgreSQL client)	Run SQL against the cluster	Use a client version equal to or newer than the server major version.
AWS CLI v2	Create, modify, back up and monitor clusters	Run `aws configure` or use SSO/instance roles. All examples assume a default Region.
IAM permissions	Allow `rds:*` actions you need, plus CloudWatch, KMS, SNS	Use least privilege; separate roles for read-only monitoring and for change operations.
Network path to the cluster	Reach port 5432	VPN, bastion host, Session Manager port-forward, or application subnet.

1.3 Conventions used

- Text in `<angle_brackets>` is a placeholder you must replace, for example `<cluster_id>`.
- Commands marked **psql** run inside the database. Commands starting with `aws` run in a shell with the AWS CLI.
- Example identifiers: cluster `prod-apg`, writer `prod-apg-writer`, reader `prod-apg-reader1`, database `appdb`.
- Example parameter group family is `aurora-postgresql17`. Replace with the family of your engine version.

> **Warning:** Aurora evolves quickly: engine versions, parameter defaults, instance classes and console screens change. Treat this handbook as a procedure template and confirm version-specific details in the Aurora User Guide and the Aurora PostgreSQL release notes before making production changes.

2. Aurora PostgreSQL Architecture for DBAs

2.1 Cluster, instances and shared storage

An Aurora PostgreSQL **DB cluster** is made of one **writer** (primary) instance, up to 15 **reader** (Aurora Replica) instances, and one **cluster volume**. Compute and storage are separated. All instances in the cluster read from the same distributed storage volume, which keeps six copies of your data across three Availability Zones. The volume grows automatically as data grows, up to 128 TiB.

- **Writer instance:** accepts reads and writes. Only one writer per cluster (unless using a Global Database secondary).
- **Reader instances:** serve read-only queries and act as failover targets. Because they share storage, replica lag is normally very small (typically well under 100 ms).
- **Storage layer:** log-structured, self-healing, quorum-based. Durability does not depend on instance-local disks.
- **Local instance storage:** used for temporary files (sorts, hash spills), logs and some caches. It is limited by instance class and is monitored with `FreeLocalStorage`.

2.2 Endpoints

Endpoint	Points to	Use for
Cluster (writer) endpoint	Current writer; changes automatically on failover	All read/write application traffic and DDL.
Reader endpoint	Load-balances (DNS round robin) across readers	Read-only application traffic, reporting.
Custom endpoint	A group of instances you choose	Isolate analytics or batch workloads on specific readers.
Instance endpoint	One specific instance	Diagnostics, per-instance monitoring. Avoid in application code.

2.3 What is different from community PostgreSQL

Area	Self-managed PostgreSQL	Aurora PostgreSQL
Superuser	`postgres` superuser	No true superuser. Use the master user and the `rds_superuser` role.
Configuration	Edit `postgresql.conf`, `pg_hba.conf`	DB cluster and DB parameter groups; access via security groups, SSL, IAM and roles.
OS access	SSH to the host	None. Use CloudWatch, Enhanced Monitoring, Performance Insights and logs.
Replication	Streaming replication, WAL shipping	Shared storage; readers added with one command. Logical replication is optional.
Backups	pg_basebackup, WAL archiving, tools such as pgBackRest	Continuous automated backups to S3, snapshots, point-in-time restore, cloning.
Checkpoint / WAL tuning	Major tuning area	Handled by the storage layer; not a normal tuning lever.
Failover	Patroni/repmgr or manual	Built in; reader promoted by promotion tier, typically within about a minute.
Storage sizing	Pre-provision and resize disks	Automatic growth; pay for what is used.
Temp space	Same disk as data	Separate local instance storage; running out causes query failures.

3. Daily, Weekly and Monthly DBA Checklists

Use these checklists as the backbone of your operating routine. Each item points to the chapter that explains the procedure. Thresholds are starting points; tune them to your workload and SLAs.

3.1 Daily checks

Check	How	Healthy / act when
Cluster and instance status	`aws rds describe-db-clusters`, console	All `available`. Investigate `failed`, `incompatible-parameters`, unexpected `pending-reboot`.
CPU and DB load	CloudWatch `CPUUtilization`, `DBLoad`; Performance Insights	CPU < 70% sustained. DBLoad below vCPU count. See Runbook R1.
Memory	`FreeableMemory`, `SwapUsage`	Freeable memory not trending to zero; swap near 0. See Runbook R12.
Connections	`DatabaseConnections` vs `max_connections`	Below 80% of the limit. See Runbook R2.
Replica lag	`AuroraReplicaLag`, `AuroraReplicaLagMaximum`	Typically < 100 ms. Alert if > 1 s sustained. See Runbook R3.
Blocked and long sessions	`pg_stat_activity` queries (Ch. 7)	No lock waits > 1 min, no idle-in-transaction > 10 min. See R5, R6.
Backups	`LatestRestorableTime`, latest automated snapshot	Latest restorable time within a few minutes of now (Ch. 11).
Log errors	CloudWatch Logs / DB log files	No new FATAL/PANIC/out-of-memory or deadlock spikes.
Transaction ID age	`MaximumUsedTransactionIDs`, `age(datfrozenxid)`	Warn at 500 million, critical at 1 billion. See Ch. 8, R8.
RDS events	`aws rds describe-events`	No unexpected failover, reboot or maintenance events.

3.2 Weekly checks

- Review top SQL by total time and by mean time (`pg_stat_statements`, Performance Insights). Open tickets for regressions.
- Review table and index growth; find unused and duplicate indexes (Chapter 9).
- Review dead tuples, last autovacuum and autoanalyze times for the largest tables (Chapter 8).
- Check replication slots for inactive or lagging slots (Chapter 15).
- Check pending maintenance actions: `aws rds describe-pending-maintenance-actions`.
- Review failed-login and permission-denied entries in logs and audit logs (Chapter 14).
- Confirm CloudWatch alarms are in OK state and notification topics still deliver.

3.3 Monthly and quarterly checks

- **Monthly:** perform a real restore test from a snapshot or point-in-time and validate data (Chapter 11).
- **Monthly:** capacity review: storage growth, instance sizing, reader count, connection peaks.
- **Monthly:** access review: list roles and privileges, remove stale users (Chapter 14).
- **Monthly:** cost review: idle clusters, old snapshots, oversized instances (Chapter 19).
- **Quarterly:** planned failover drill in a staging or low-risk window (Chapter 12).
- **Quarterly:** minor and major version roadmap; extension version review; certificate (CA) expiry review (Chapter 13).
- **Quarterly:** disaster-recovery exercise, including cross-Region restore or Global Database switchover.

4. Connecting and Basic Access

4.1 Connect with psql

1. Find the endpoint: `aws rds describe-db-clusters --db-cluster-identifier prod-apg --query "DBClusters[0].[Endpoint,ReaderEndpoint,Port]"`
2. Download the Amazon RDS certificate bundle (global bundle) from the RDS trust store and keep it on the client.
3. Connect with certificate verification enabled.

```
# Writer
psql "host=prod-apg.cluster-xxxx.us-east-1.rds.amazonaws.com port=5432 dbname=appdb \
      user=dba_admin sslmode=verify-full sslrootcert=/etc/ssl/rds/global-bundle.pem"

# Reader (read-only)
psql "host=prod-apg.cluster-ro-xxxx.us-east-1.rds.amazonaws.com port=5432 dbname=appdb \
      user=app_ro sslmode=verify-full sslrootcert=/etc/ssl/rds/global-bundle.pem"
```

4.2 Identify where you are connected

```
-- psql
SELECT version();
SELECT aurora_version();
SELECT aurora_db_instance_identifier();      -- which instance am I on?
SELECT pg_is_in_recovery();                  -- false = writer, true = reader
SELECT current_user, session_user, current_database(), inet_server_addr();
```

4.3 Essential psql commands

Command	Meaning
`\l` / `\c db`	List databases / connect to a database
`\dt+` / `\di+`	List tables / indexes with sizes
`\du+`	List roles and attributes
`\dx`	List installed extensions
`\d+ table`	Describe a table including indexes, constraints, storage
`\x auto`	Expanded display for wide rows
`\timing`	Show execution time of each statement
`\watch 5`	Re-run the last query every 5 seconds
`\conninfo`	Show connection details

4.4 Cannot connect: step-by-step diagnosis

1. **DNS:** `nslookup <endpoint>` resolves to a private IP from inside the VPC. If it returns a public IP, check whether the cluster is publicly accessible.
2. **Network:** `nc -vz <endpoint> 5432` (or `Test-NetConnection` on Windows). A timeout means routing, NACL or security group problems.
3. **Security group:** the cluster security group must allow inbound TCP 5432 from the client IP range or from the application security group.
4. **Subnet routing / NACL:** verify route tables and network ACLs allow return traffic.
5. **Authentication:** error `password authentication failed` means wrong password or auth method (password vs IAM). Check `password_encryption` and the role setting.
6. **SSL:** error `no pg_hba.conf entry ... no encryption` means the server requires SSL (`rds.force_ssl=1`); connect with `sslmode=require` or stricter.

7. **Capacity:** error `too many connections` or `remaining connection slots are reserved` means connection exhaustion (Runbook R2).
8. **Instance state:** the instance may be rebooting or failing over. Check `describe-events` for the last hour.

5. Provisioning a Cluster (Step by Step)

5.1 Pre-requisites

- A VPC with a **DB subnet group** that spans at least two (ideally three) Availability Zones.
- A **security group** that allows port 5432 only from application security groups and administration hosts.
- A **KMS key** for storage encryption (the AWS managed key `aws/rds` is acceptable; a customer managed key gives more control).
- An SNS topic for alarms and event notifications.
- Agreed naming, tagging (owner, environment, cost-center) and backup retention policy.

5.2 Step 1: create parameter groups

```
aws rds create-db-cluster-parameter-group \
  --db-cluster-parameter-group-name prod-apg17-cluster \
  --db-parameter-group-family aurora-postgresql17 \
  --description "Prod Aurora PG17 cluster params"

aws rds create-db-parameter-group \
  --db-parameter-group-name prod-apg17-instance \
  --db-parameter-group-family aurora-postgresql17 \
  --description "Prod Aurora PG17 instance params"
```

5.3 Step 2: choose the engine version

```
aws rds describe-db-engine-versions --engine aurora-postgresql \
  --query "DBEngineVersions[].EngineVersion" --output text

# Valid upgrade targets for a given version
aws rds describe-db-engine-versions --engine aurora-postgresql --engine-version <current_version> \
  --query "DBEngineVersions[].ValidUpgradeTarget[].EngineVersion"
```

Prefer the newest major version that your extensions and application drivers support, and a minor version that is still under standard support. Check the Aurora PostgreSQL release notes for the support calendar.

5.4 Step 3: create the cluster

```
aws rds create-db-cluster \
  --db-cluster-identifier prod-apg \
  --engine aurora-postgresql --engine-version <ENGINE_VERSION> \
  --master-username dba_master --manage-master-user-password \
  --db-subnet-group-name prod-db-subnets \
  --vpc-security-group-ids sg-0123456789abcdef0 \
  --db-cluster-parameter-group-name prod-apg17-cluster \
  --storage-encrypted --kms-key-id alias/prod-rds \
  --backup-retention-period 14 \
  --preferred-backup-window 21:00-21:30 \
  --preferred-maintenance-window sun:22:00-sun:22:30 \
  --enable-cloudwatch-logs-exports postgresql \
  --deletion-protection \
  --copy-tags-to-snapshot \
  --tags Key=env,Value=prod Key=owner,Value=dba-team
```

> **Tip:** `--manage-master-user-password` stores and rotates the master password in AWS Secrets Manager. To set a password yourself use `--master-user-password` instead, but store it in a secret manager, never in scripts.

5.5 Step 4: add the writer and readers

```
aws rds create-db-instance \
  --db-instance-identifier prod-apg-writer --db-cluster-identifier prod-apg \
  --engine aurora-postgresql --db-instance-class db.r6g.xlarge \
  --db-parameter-group-name prod-apg17-instance \
  --monitoring-interval 15 --monitoring-role-arn arn:aws:iam::<acct>:role/rds-monitoring-role \
  --enable-performance-insights --performance-insights-retention-period 7 \
  --promotion-tier 0 --availability-zone us-east-1a

aws rds create-db-instance \
  --db-instance-identifier prod-apg-reader1 --db-cluster-identifier prod-apg \
  --engine aurora-postgresql --db-instance-class db.r6g.xlarge \
  --db-parameter-group-name prod-apg17-instance \
  --monitoring-interval 15 --monitoring-role-arn arn:aws:iam::<acct>:role/rds-monitoring-role \
  --enable-performance-insights --promotion-tier 1 --availability-zone us-east-1b
```

Place the writer and at least one reader in different Availability Zones. The first instance created in a new cluster becomes the writer. Use the same instance class for readers that must be able to take over the writer role.

5.6 Step 5: verify

```
aws rds describe-db-clusters --db-cluster-identifier prod-apg \
  --query
      "DBClusters[0].[Status,Endpoint,ReaderEndpoint,DBClusterMembers[].[DBInstanceIdentifier,IsClusterWriter]]"
aws rds describe-db-instances --filters Name=db-cluster-id,Values=prod-apg \
  --query "DBInstances[].[DBInstanceIdentifier,DBInstanceStatus,DBInstanceClass,AvailabilityZone]"
      --output table
```

5.7 Step 6: post-creation baseline

1. Connect as the master user and create application roles and databases (Chapter 14).
2. Install required extensions, for example `pg_stat_statements` (Chapter 18).
3. Create CloudWatch alarms (Chapter 7) and an RDS event subscription.
4. Confirm automated backups and test restore (Chapter 11).
5. Document endpoints, secrets location, parameter groups and the on-call runbook links.

5.8 Choosing instance classes

Workload	Typical choice	Comment
Dev/test, intermittent	Aurora Serverless v2 (`db.serverless`) or small burstable classes	Serverless v2 scales in fine increments of ACUs between your min and max.
General OLTP	Memory-optimized `db.r*` (Graviton g types are usually best price/performance)	Size so the hot working set fits in memory; watch BufferCacheHitRatio.
I/O-intensive, large temp usage	Classes with local NVMe (Aurora Optimized Reads) where available	Verify class and version support in the docs.
Spiky/unpredictable	Serverless v2 for readers, or a mixed cluster	Set a minimum ACU high enough to keep the cache warm.

```
# Serverless v2 example: set scaling range on the cluster, then add db.serverless instances
aws rds modify-db-cluster --db-cluster-identifier prod-apg \
  --serverless-v2-scaling-configuration MinCapacity=2,MaxCapacity=32
aws rds create-db-instance --db-instance-identifier prod-apg-sl1 --db-cluster-identifier prod-apg \
  --engine aurora-postgresql --db-instance-class db.serverless
```

6. Parameter Management

6.1 Two levels of parameter groups

- **DB cluster parameter group:** settings shared by all instances (for example `rds.force_ssl`, `rds.logical_replication`, `shared_preload_libraries`, autovacuum settings, `apg_ccm_enabled`).
- **DB parameter group:** settings that can differ per instance (for example `work_mem`, `log_min_duration_statement`, instance-specific limits). If a parameter exists in both, the instance-level value can override the cluster one; check with `SHOW <name>`.
- **Dynamic parameters** apply immediately. **Static parameters** need a reboot (status `pending-reboot`).

6.2 Change a parameter (procedure)

1. Never edit the default parameter group. Use a custom group (Section 5.2).
2. Change the value; choose `immediate` for dynamic and `pending-reboot` for static parameters.
3. Check the status of the instances and reboot in a maintenance window when needed (fail over first so the writer reboots last).
4. Verify with `SHOW` inside the database.

```
aws rds modify-db-cluster-parameter-group --db-cluster-parameter-group-name prod-apg17-cluster \
  --parameters "ParameterName=rds.force_ssl,ParameterValue=1,ApplyMethod=immediate"

aws rds modify-db-parameter-group --db-parameter-group-name prod-apg17-instance \
  --parameters "ParameterName=log_min_duration_statement,ParameterValue=1000,ApplyMethod=immediate"
      \
               "ParameterName=track_io_timing,ParameterValue=1,ApplyMethod=immediate"

# Find parameters that differ from default / are pending
aws rds describe-db-parameters --db-parameter-group-name prod-apg17-instance --source user
aws rds describe-db-instances --db-instance-identifier prod-apg-writer \
  --query "DBInstances[0].DBParameterGroups"
-- psql
SELECT name, setting, unit, source, pending_restart FROM pg_settings WHERE name =
    'log_min_duration_statement';
```

6.3 Recommended baseline parameters

Parameter	Suggested value	Why
`shared_preload_libraries`	`pg_stat_statements` (add `pgaudit`, `pg_cron`, `pg_hint_plan` if used)	Static (reboot). Query statistics and optional tools.
`pg_stat_statements.track`	`all` or `top`	Collect SQL statistics.
`track_io_timing`	`1`	I/O timing in EXPLAIN and pg_stat_statements.
`log_min_duration_statement`	`1000` ms (lower in tuning periods)	Slow query log.
`log_lock_waits`	`1`	Log waits longer than `deadlock_timeout`.
`log_temp_files`	`10240` (kB)	Log queries that spill to disk.
`log_autovacuum_min_duration`	`1000` ms (or 0 temporarily)	See what autovacuum does.
`log_connections` / `log_disconnections`	`1` if audit requires; otherwise off	Can be noisy on busy systems.
`idle_in_transaction_session_timeout`	`600000` ms (10 min), tune per app	Stops sessions that hold locks and block vacuum.
`statement_timeout`	Set per role/app, not globally	Protects from runaway queries.
`lock_timeout`	Set in migration sessions (for example 5s)	Avoid DDL lock pile-ups.
`rds.force_ssl`	`1`	Require TLS.
`password_encryption`	`scram-sha-256`	Stronger password hashing.
`rds.adaptive_autovacuum`	`1`	Aurora/RDS boosts autovacuum automatically when wraparound approaches.
`rds.log_retention_period`	`4320` min default (3 days)	Logs live on local storage; shorten if local storage is tight.
`apg_ccm_enabled`	`1` (cluster level)	Cluster cache management: warm cache on the failover target (tier-0 reader).

6.4 Connections and memory

- `max_connections` default is derived from instance memory (formula `LEAST({DBInstanceClassMemory/9531392}, 5000)`). More connections use more memory; prefer a connection pooler over raising the limit.
- `shared_buffers` is set automatically to a large share of instance memory (roughly 75%). Leave it unless AWS guidance for your case says otherwise.
- `work_mem` is allocated **per sort/hash node per query**, not per session. A safe starting point is 4-16 MB; raise for specific roles or sessions (`SET work_mem = ...` or `ALTER ROLE report_user SET work_mem = ...`), not globally.
- `maintenance_work_mem` speeds `CREATE INDEX` and `VACUUM`; raise it in the session that runs maintenance.
- `temp_file_limit` protects local storage from runaway queries.

> **Warning:** Changing a global memory parameter is the most common cause of out-of-memory restarts. Change one parameter at a time, record the before/after in your change log, and watch `FreeableMemory` and `SwapUsage`.

7. Monitoring and Alerting

7.1 Key CloudWatch metrics

Metric (AWS/RDS)	What it tells you	Starting alarm threshold
`CPUUtilization`	Instance CPU	> 80% for 15 min
`DBLoad` (Performance Insights)	Average active sessions	> vCPU count for 10 min
`FreeableMemory`	Memory available to the OS and cache	< 10-15% of instance RAM
`SwapUsage`	Swap in use (bad sign)	> 100 MB sustained
`DatabaseConnections`	Open connections	> 80% of `max_connections`
`AuroraReplicaLag` / `AuroraReplicaLagMaximum`	Reader lag behind writer (ms)	> 1000 ms for 5 min
`FreeLocalStorage`	Local disk for temp files and logs	< 20% of initial, or < 10 GB
`VolumeBytesUsed`	Cluster volume size	Trend alarm / sudden growth
`MaximumUsedTransactionIDs`	Oldest transaction ID age	> 500 million warn, > 1 billion critical
`BufferCacheHitRatio`	Cache efficiency	Sustained drop below your baseline (often < 95%)
`CommitLatency`, `ReadLatency`, `WriteLatency`	Storage and commit latency	Alert on deviation from baseline
`Deadlocks`	Deadlock count	> 0 sustained
`TransactionLogsDiskUsage`	Space used by transaction logs (important with logical slots)	Sudden growth
`OldestReplicationSlotLag`	Lag of the oldest logical slot	Growing continuously
`ServerlessDatabaseCapacity`	ACUs in use (Serverless v2)	Near max capacity

7.2 Create an alarm

```
aws cloudwatch put-metric-alarm --alarm-name prod-apg-writer-cpu-high \
  --namespace AWS/RDS --metric-name CPUUtilization \
  --dimensions Name=DBInstanceIdentifier,Value=prod-apg-writer \
  --statistic Average --period 300 --evaluation-periods 3 --threshold 80 \
  --comparison-operator GreaterThanThreshold \
  --alarm-actions arn:aws:sns:us-east-1:<acct>:dba-alerts \
  --ok-actions   arn:aws:sns:us-east-1:<acct>:dba-alerts --treat-missing-data notBreaching

# Cluster-level metric (use the DBClusterIdentifier dimension), example: volume size
aws cloudwatch put-metric-alarm --alarm-name prod-apg-volume-growth \
  --namespace AWS/RDS --metric-name VolumeBytesUsed \
  --dimensions Name=DBClusterIdentifier,Value=prod-apg \
  --statistic Maximum --period 3600 --evaluation-periods 1 --threshold 8796093022208 \
  --comparison-operator GreaterThanThreshold --alarm-actions arn:aws:sns:us-east-1:<acct>:dba-alerts
```

7.3 Enhanced Monitoring and Performance Insights

```
aws rds modify-db-instance --db-instance-identifier prod-apg-writer --apply-immediately \
  --monitoring-interval 15 --monitoring-role-arn arn:aws:iam::<acct>:role/rds-monitoring-role \
  --enable-performance-insights --performance-insights-retention-period 7
```

Enhanced Monitoring shows OS-level metrics (per-process CPU and memory, load, swap) at an interval you choose from 1 to 60 seconds. **Performance Insights** (being integrated into **CloudWatch Database Insights**) charts database load as average active sessions and breaks it down by wait event, SQL statement, user, host and database. Check the current console naming and retention options in the AWS documentation.

How to read Performance Insights in five steps

1. Set the time range to cover the incident. Compare DB load with the **max vCPU** line. Load above the line means sessions are waiting.
2. Slice by **Wait events**. The dominant colour tells you the class of problem (CPU, I/O, lock, client).
3. Slice by **SQL** to find the top statements for that wait event.
4. Slice by **User/Host/Database** to identify the application or batch job.
5. Take the SQL text to `EXPLAIN (ANALYZE, BUFFERS)` (Chapter 9) and decide: index, rewrite, statistics, or workload change.

Wait event	Meaning	Typical action
`CPU`	Sessions running or waiting for CPU	Find top SQL, add indexes, reduce parallelism, scale up.
`IO:DataFileRead`	Reading data pages from storage (cache misses)	Add indexes, reduce scans, increase instance memory, check cache hit ratio.
`IO:XactSync`	Waiting for commit to be durable in storage	Batch commits, reduce commit rate; check network/AZ latency.
`Lock:transactionid`, `Lock:tuple`	Row-level contention	Shorten transactions, fix hot rows, consistent update order.
`Lock:relation`	Table-level lock (DDL or LOCK TABLE) waiting	Find blocker (Ch. 10); use `lock_timeout` for DDL.
`LWLock:buffer_content`	Contention on a hot page or index leaf	Reduce hot-spot inserts, partitioning, review index design.
`LWLock:lock_manager`	Many relations or partitions locked per query	Reduce partitions per query, prepared statements, pruning.
`Client:ClientRead`	Server waiting for the client	Slow application, idle-in-transaction; fix client side.

7.4 Logs

```
# Publish PostgreSQL logs to CloudWatch Logs
aws rds modify-db-cluster --db-cluster-identifier prod-apg --apply-immediately \
  --cloudwatch-logs-export-configuration '{"EnableLogTypes":["postgresql"]}'

# Download a log file portion from an instance
aws rds describe-db-log-files --db-instance-identifier prod-apg-writer
aws rds download-db-log-file-portion --db-instance-identifier prod-apg-writer \
  --log-file-name error/postgresql.log.2026-10-27-10 --output text > pg.log
```

7.5 In-database monitoring queries (psql)

Active sessions and what they wait on

```
SELECT pid, usename, application_name, client_addr, state, wait_event_type, wait_event,
       now() - xact_start AS xact_age, now() - query_start AS query_age, left(query, 120) AS query
FROM pg_stat_activity
WHERE state <> 'idle' AND pid <> pg_backend_pid()
ORDER BY query_start;
```

Connections by state, user and application

```
SELECT usename, application_name, state, count(*)
FROM pg_stat_activity GROUP BY 1,2,3 ORDER BY 4 DESC;
SHOW max_connections;
```

Long-running queries and transactions

```
SELECT pid, usename, state, now() - xact_start AS xact_age, now() - query_start AS query_age,
    left(query,100) q
FROM pg_stat_activity
WHERE (now() - xact_start) > interval '5 minutes' OR (state = 'idle in transaction'
      AND now() - state_change > interval '5 minutes')
ORDER BY xact_start;
```

Database size, cache hit ratio, temp usage, deadlocks

```
SELECT datname, pg_size_pretty(pg_database_size(datname)) size,
       round(100.0*blks_hit/nullif(blks_hit+blks_read,0),2) AS cache_hit_pct,
       temp_files, pg_size_pretty(temp_bytes) temp_bytes, deadlocks, xact_commit, xact_rollback
FROM pg_stat_database WHERE datname NOT IN ('template0','template1','rdsadmin');
```

Reader status from the writer

```
SELECT server_id, session_id, replica_lag_in_msec, last_update_timestamp
FROM aurora_replica_status() ORDER BY server_id;
-- session_id = 'MASTER_SESSION_ID' is the writer
```

7.6 pg_stat_statements: find your most expensive SQL

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;   -- after it is in shared_preload_libraries +
    reboot

-- Top 15 by total time
SELECT queryid, calls, round(total_exec_time::numeric,1) total_ms, round(mean_exec_time::numeric,2)
    mean_ms,
       rows, shared_blks_read, temp_blks_written, left(query, 100) query
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 15;

-- Slowest on average with at least 20 calls
SELECT queryid, calls, round(mean_exec_time::numeric,2) mean_ms, round(stddev_exec_time::numeric,2)
    sd_ms, left(query,100) q
FROM pg_stat_statements WHERE calls >= 20 ORDER BY mean_exec_time DESC LIMIT 15;

-- Reset (do this after a major change so you measure fresh data)
SELECT pg_stat_statements_reset();
```

7.7 Event notifications

```
aws rds create-event-subscription --subscription-name prod-apg-events \
  --sns-topic-arn arn:aws:sns:us-east-1:<acct>:dba-alerts --source-type db-cluster \
  --event-categories failover failure maintenance notification --source-ids prod-apg
aws rds create-event-subscription --subscription-name prod-apg-inst-events \
  --sns-topic-arn arn:aws:sns:us-east-1:<acct>:dba-alerts --source-type db-instance \
  --event-categories availability failure low-storage maintenance recovery "configuration change"
aws rds describe-events --source-type db-cluster --source-identifier prod-apg --duration 1440
```

8. Vacuum, Bloat and Transaction ID Wraparound

PostgreSQL uses MVCC: updates and deletes leave old row versions (dead tuples) that autovacuum must clean. Neglected vacuuming causes table and index **bloat**, slow queries, and in the worst case **transaction ID wraparound**, which can force the database into a protective read-only state. This is the most important DBA topic in PostgreSQL.

8.1 Daily health queries

Dead tuples and autovacuum recency

```
SELECT schemaname, relname, n_live_tup, n_dead_tup,
       round(100.0*n_dead_tup/nullif(n_live_tup+n_dead_tup,0),1) AS dead_pct,
       last_autovacuum, last_autoanalyze, autovacuum_count
FROM pg_stat_user_tables ORDER BY n_dead_tup DESC LIMIT 20;
```

Transaction ID age (database and table level)

```
SELECT datname, age(datfrozenxid) AS xid_age,
       round(100.0*age(datfrozenxid)/2147483647,1) AS pct_to_wraparound
FROM pg_database ORDER BY 2 DESC;

SELECT c.oid::regclass AS tbl, age(c.relfrozenxid) AS xid_age,
       pg_size_pretty(pg_total_relation_size(c.oid)) AS size
FROM pg_class c JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind IN ('r','m','t') AND n.nspname NOT IN ('pg_catalog','information_schema')
ORDER BY age(c.relfrozenxid) DESC LIMIT 20;
```

Running autovacuum workers and vacuum progress

```
SELECT pid, now()-xact_start AS runtime, query FROM pg_stat_activity WHERE query LIKE
    'autovacuum:%';
SELECT p.pid, p.relid::regclass, p.phase, p.heap_blks_total, p.heap_blks_scanned,
    p.heap_blks_vacuumed
FROM pg_stat_progress_vacuum p;
```

8.2 What blocks vacuum from cleaning (check all four)

```
-- 1) Oldest open transactions
SELECT pid, usename, state, age(backend_xid) xid_age, age(backend_xmin) xmin_age, now()-xact_start
    AS xact_age,
       left(query,80) q
FROM pg_stat_activity WHERE backend_xmin IS NOT NULL ORDER BY age(backend_xmin) DESC LIMIT 10;

-- 2) Orphaned prepared transactions
SELECT gid, prepared, owner, database FROM pg_prepared_xacts ORDER BY prepared;

-- 3) Replication slots holding back xmin / catalog_xmin
SELECT slot_name, slot_type, active, xmin, catalog_xmin, restart_lsn FROM pg_replication_slots;

-- 4) Long queries on readers can also delay cleanup or cause conflicts; check pg_stat_activity on
    readers.
```

8.3 Tune autovacuum

Defaults (`autovacuum_vacuum_scale_factor = 0.2`) wait until 20% of a table changes. That is too late for large tables. Tune per table for the big, busy tables and keep global defaults reasonable.

```
-- Per-table settings for a large, busy table
ALTER TABLE public.orders SET (
  autovacuum_vacuum_scale_factor = 0.02,
  autovacuum_analyze_scale_factor = 0.01,
  autovacuum_vacuum_threshold = 1000,
  autovacuum_vacuum_cost_limit = 2000);

-- Check per-table overrides
SELECT relname, reloptions FROM pg_class WHERE reloptions IS NOT NULL;
```

Parameter (cluster group)	Guidance
`autovacuum_max_workers`	Raise for many large tables (static, reboot). Each worker uses `autovacuum_work_mem`.
`autovacuum_naptime`	Default 15 s is fine; lower only for very high churn.
`autovacuum_vacuum_cost_limit` / `_cost_delay`	Raise the limit (or lower delay) so vacuum keeps up with write rate.
`autovacuum_work_mem` / `maintenance_work_mem`	More memory means fewer index passes on large tables.
`autovacuum_freeze_max_age`	Default 200 million: when reached, aggressive anti-wraparound vacuum starts. Do not raise without need.
`rds.adaptive_autovacuum`	Keep at 1.

8.4 Manual vacuum and analyze

```
VACUUM (VERBOSE, ANALYZE) public.orders;                -- routine, does not block normal
    reads/writes
VACUUM (FREEZE, VERBOSE) public.orders;                 -- aggressive freeze
SET maintenance_work_mem = '2GB'; VACUUM (INDEX_CLEANUP ON, PARALLEL 4, VERBOSE) public.big_table;
ANALYZE public.orders;                                  -- refresh planner statistics
-- Whole database from shell (parallel), useful after major upgrade:
--   vacuumdb --host=<endpoint> --username=<user> --dbname=appdb --analyze-in-stages --jobs=4
```

8.5 Measure and fix bloat

1. Measure with `pgstattuple` (exact, scans table) or `pgstattuple_approx` (fast, approximate).
2. Decide: bloat under about 20-30% on small tables is usually ignored. Large bloated tables and indexes need action.
3. Fix indexes online with `REINDEX INDEX CONCURRENTLY` (PostgreSQL 12 and later).
4. Fix tables online with `pg_repack` (needs a primary key or unique index and extra free space).
5. `VACUUM FULL` rewrites the table under an **ACCESS EXCLUSIVE lock**. Use only in a maintenance window.
6. Fix the cause: tune autovacuum, shorten transactions, use `fillfactor` for update-heavy tables to enable HOT updates.

```
CREATE EXTENSION IF NOT EXISTS pgstattuple;
SELECT * FROM pgstattuple_approx('public.orders');          -- check dead_tuple_percent,
    approx_free_percent
SELECT * FROM pgstatindex('public.orders_pkey');            -- avg_leaf_density, leaf_fragmentation

REINDEX INDEX CONCURRENTLY public.orders_customer_idx;
ALTER TABLE public.orders SET (fillfactor = 90);

-- pg_repack (extension must be installed in the database; run client from a host that can reach the
    cluster)
CREATE EXTENSION pg_repack;
--   pg_repack --host=<endpoint> --username=<user> --dbname=appdb --table=public.orders --no-order
```

> **Note:** Aurora releases storage automatically when you drop or truncate whole tables and indexes, but space freed by DELETE inside a table is only reused by that table until it is rewritten (pg_repack or VACUUM FULL).

8.6 Emergency: wraparound protection

When `age(datfrozenxid)` approaches 2 billion, PostgreSQL stops accepting new write transactions. Follow Runbook R8 (Chapter 17) immediately if the age is above 1 billion and rising or the log shows warnings about wraparound.

9. Performance Tuning Workflow

9.1 Method

1. **Define the symptom:** which users, which time, which metric (latency, throughput, errors).
2. **Find where time goes:** Performance Insights wait events and top SQL, `pg_stat_statements`.
3. **Inspect the plan:** `EXPLAIN (ANALYZE, BUFFERS)` on a realistic copy (a clone; see Chapter 11).
4. **Fix one thing:** index, query rewrite, statistics, parameter, pooling, hardware.
5. **Measure again** and keep a before/after record. Reset statistics if needed.

9.2 Reading EXPLAIN

```
EXPLAIN (ANALYZE, BUFFERS, VERBOSE, SETTINGS)
SELECT o.* FROM orders o WHERE o.customer_id = 42 AND o.created_at >= now() - interval '7 days';
```

Plan symptom	Meaning	Typical fix
Seq Scan on a large table with a selective filter	No usable index	Create an index on the filter columns; check for functions on the column.
Rows estimated vs actual differ by 10x or more	Stale or insufficient statistics	ANALYZE; raise `default_statistics_target`; create extended statistics.
`Sort Method: external merge Disk`	work_mem too small for this sort	Raise `work_mem` for that role/session; add index to avoid sort.
`Hash Batches > 1`	Hash join spilled	Raise `work_mem`; reduce rows earlier.
Nested Loop with huge inner loops	Bad join order or estimate	Update stats; index the join key; rewrite; consider plan management.
High `shared read` in BUFFERS	Data came from storage, not cache	Better index, more memory, warm the cache.
Heap Fetches high in Index Only Scan	Visibility map outdated	Vacuum the table more often.

9.3 Index management

Find unused indexes (review over a full business cycle before dropping)

```
SELECT s.schemaname, s.relname, s.indexrelname, s.idx_scan,
    pg_size_pretty(pg_relation_size(s.indexrelid)) AS size
FROM pg_stat_user_indexes s JOIN pg_index i ON i.indexrelid = s.indexrelid
WHERE s.idx_scan = 0 AND NOT i.indisunique AND NOT i.indisprimary
ORDER BY pg_relation_size(s.indexrelid) DESC LIMIT 20;
```

Find tables with many sequential scans (candidates for new indexes)

```
SELECT relname, seq_scan, seq_tup_read, idx_scan, n_live_tup,
    pg_size_pretty(pg_relation_size(relid)) size
FROM pg_stat_user_tables WHERE seq_scan > 100 AND n_live_tup > 100000
ORDER BY seq_tup_read DESC LIMIT 20;
```

Find duplicate indexes and invalid indexes

```
SELECT indrelid::regclass AS tbl, array_agg(indexrelid::regclass) AS dup_indexes
FROM pg_index GROUP BY indrelid, indkey, indclass, indpred::text, indexprs::text HAVING count(*) >
    1;

SELECT indexrelid::regclass, indrelid::regclass FROM pg_index WHERE NOT indisvalid;   -- failed
    concurrent builds
```

Create and drop indexes safely

```
SET lock_timeout = '5s';
CREATE INDEX CONCURRENTLY idx_orders_cust_created ON public.orders (customer_id, created_at DESC);
-- If it fails, drop the invalid index and retry:
DROP INDEX CONCURRENTLY IF EXISTS public.idx_orders_cust_created;
```

9.4 Statistics

```
ALTER TABLE public.orders ALTER COLUMN customer_id SET STATISTICS 500;
CREATE STATISTICS orders_cust_status (dependencies) ON customer_id, status FROM public.orders;
ANALYZE public.orders;
```

9.5 Connection management

- Applications should use a **connection pool** (application-side such as HikariCP, or PgBouncer, or **Amazon RDS Proxy**). Each PostgreSQL connection is a process that consumes memory.
- Pool size guidance: start near `2 x vCPU` of the writer across all app instances, then measure. More connections rarely mean more throughput.
- Use the **reader endpoint** for read-only work. Keep session-state-heavy traffic (temp tables, advisory locks) on a single instance.
- Set `idle_in_transaction_session_timeout` and client keepalives so dead clients release resources.
- For serverless or Lambda-style workloads, put RDS Proxy in front of the cluster to absorb connection churn and speed failover.

9.6 Plan stability

- **Query Plan Management (**`apg_plan_mgmt` **extension)** captures and approves query plans to prevent regressions after upgrades or statistics changes.
- `pg_hint_plan` lets you force a plan shape with hints for a specific query as a last resort.
- After a major version upgrade, run `ANALYZE` and compare `pg_stat_statements` before and after.

9.7 Scale reads and use the cache wisely

- Add readers and route reporting traffic to a custom endpoint so it does not compete with OLTP readers.
- Use Aurora Auto Scaling for reader count (Chapter 12).
- Enable cluster cache management (`apg_ccm_enabled`) so the failover target has a warm cache.

10. Locks, Blocking and Deadlocks

10.1 Find who is blocking whom

```
SELECT blocked.pid AS blocked_pid, blocked.usename AS blocked_user, now()-blocked.query_start AS
    waiting_for,
       left(blocked.query,80) AS blocked_query,
       blocker.pid AS blocker_pid, blocker.usename AS blocker_user, blocker.state AS blocker_state,
       now()-blocker.xact_start AS blocker_xact_age, left(blocker.query,80) AS blocker_query
FROM pg_stat_activity blocked
JOIN LATERAL unnest(pg_blocking_pids(blocked.pid)) AS b(pid) ON true
JOIN pg_stat_activity blocker ON blocker.pid = b.pid
ORDER BY waiting_for DESC;
```

10.2 Resolve a blocking incident

1. Identify the **root blocker** (a blocker that is itself not blocked). Often it is `idle in transaction`.
2. Ask: is it a critical job? If it is an abandoned session, cancel the query first: `SELECT pg_cancel_backend(<pid>);`
3. If the session stays open, terminate it: `SELECT pg_terminate_backend(<pid>);` (rolls back its transaction).
4. Confirm the queue drains by re-running the blocking query.
5. Prevent recurrence: `idle_in_transaction_session_timeout`, shorter transactions, `lock_timeout` for DDL, app fixes.

10.3 Safe DDL practice on busy tables

```
-- Always bound waiting; retry in the application or script if it times out
SET lock_timeout = '3s';
SET statement_timeout = '15min';
ALTER TABLE public.orders ADD COLUMN note text;                          -- fast in modern PostgreSQL
    (no rewrite)
ALTER TABLE public.orders ADD CONSTRAINT fk_cust FOREIGN KEY (customer_id)
  REFERENCES public.customers(id) NOT VALID;                           -- no long scan under lock
ALTER TABLE public.orders VALIDATE CONSTRAINT fk_cust;                  -- weaker lock
```

10.4 Deadlocks

- Ensure `log_lock_waits = 1` and review the log: the deadlock message lists the statements and processes involved.
- Fix in the application: access rows and tables in a consistent order, keep transactions short, avoid user think-time inside a transaction.
- Monitor the `Deadlocks` CloudWatch metric and `pg_stat_database.deadlocks`.

11. Backup, Restore and Cloning

11.1 How Aurora backup works

- **Automated continuous backup** to S3 with retention of 1 to 35 days. This enables point-in-time restore (PITR) to any second within the window.
- **Manual snapshots** are kept until you delete them. Use for pre-change safety, long-term retention and cross-Region copies.
- A restore **always creates a new cluster**. You then add instances to it and repoint applications.
- **Cloning** creates a new cluster that shares storage pages with the source (copy-on-write): fast and space-efficient, ideal for testing and troubleshooting.
- **AWS Backup** can manage schedules, long retention, vaults and cross-account/Region copies centrally.
- Aurora PostgreSQL does not support Backtrack (that feature exists only for Aurora MySQL). Use PITR or cloning instead.

11.2 Check backup status

```
aws rds describe-db-clusters --db-cluster-identifier prod-apg \
  --query "DBClusters[0].[BackupRetentionPeriod,LatestRestorableTime,EarliestRestorableTime]"
aws rds describe-db-cluster-snapshots --db-cluster-identifier prod-apg \
  --query
      "DBClusterSnapshots[].[DBClusterSnapshotIdentifier,SnapshotType,SnapshotCreateTime,Status]"
      --output table
```

11.3 Procedure A: manual snapshot (before every risky change)

```
aws rds create-db-cluster-snapshot --db-cluster-identifier prod-apg \
  --db-cluster-snapshot-identifier prod-apg-pre-change-20261027
aws rds wait db-cluster-snapshot-available --db-cluster-snapshot-identifier
    prod-apg-pre-change-20261027
```

11.4 Procedure B: restore from a snapshot

1. Restore the cluster (this creates the cluster only, with no instances).
2. Add at least one instance to the restored cluster.
3. Verify: connect, check row counts and application smoke tests.
4. Switch the application or rename endpoints in a planned cutover; keep the old cluster until validation is done.

```
aws rds restore-db-cluster-from-snapshot --db-cluster-identifier prod-apg-restored \
  --snapshot-identifier prod-apg-pre-change-20261027 --engine aurora-postgresql \
  --db-subnet-group-name prod-db-subnets --vpc-security-group-ids sg-0123456789abcdef0 \
  --db-cluster-parameter-group-name prod-apg17-cluster --kms-key-id alias/prod-rds \
  --deletion-protection --enable-cloudwatch-logs-exports postgresql
aws rds create-db-instance --db-instance-identifier prod-apg-restored-w --db-cluster-identifier
    prod-apg-restored \
  --engine aurora-postgresql --db-instance-class db.r6g.xlarge --db-parameter-group-name
      prod-apg17-instance
```

Warning: A restored cluster does not automatically inherit custom parameter groups, security groups, monitoring settings or alarms. Specify them in the restore command and recreate alarms.

11.5 Procedure C: point-in-time restore (PITR)

```
# Restore to a specific UTC time
aws rds restore-db-cluster-to-point-in-time --source-db-cluster-identifier prod-apg \
  --db-cluster-identifier prod-apg-pitr --restore-to-time 2026-10-27T10:15:00Z \
  --db-subnet-group-name prod-db-subnets --vpc-security-group-ids sg-0123456789abcdef0 \
  --db-cluster-parameter-group-name prod-apg17-cluster

# Or restore to the latest restorable time
aws rds restore-db-cluster-to-point-in-time --source-db-cluster-identifier prod-apg \
  --db-cluster-identifier prod-apg-pitr --use-latest-restorable-time \
  --db-subnet-group-name prod-db-subnets
# Then create an instance in prod-apg-pitr (see Procedure B step 2)
```

11.6 Procedure D: clone for testing

```
aws rds restore-db-cluster-to-point-in-time --source-db-cluster-identifier prod-apg \
  --db-cluster-identifier prod-apg-clone --restore-type copy-on-write --use-latest-restorable-time \
  --db-subnet-group-name prod-db-subnets --vpc-security-group-ids sg-0123456789abcdef0
aws rds create-db-instance --db-instance-identifier prod-apg-clone-w --db-cluster-identifier
    prod-apg-clone \
  --engine aurora-postgresql --db-instance-class db.r6g.large
```

Use clones to rehearse major upgrades, reproduce slow queries with `EXPLAIN ANALYZE`, and test migrations. Mask or protect any sensitive data in clones; they contain a full copy of production data. Delete clones when finished.

11.7 Procedure E: recover from an accidental DROP/DELETE

1. **Stop the damage:** if data loss is ongoing, pause the offending job or application. Note the exact UTC time of the mistake.
2. Run a **PITR** (or clone) to a time just before the incident into a new cluster (Procedures C/D). Do not overwrite production.
3. Add an instance to the new cluster and connect.
4. Export only what you need: `pg_dump --table=public.orders --data-only` from the restored cluster, or use `COPY ... TO`.
5. Load into production in a controlled way (staging table first, then `INSERT ... SELECT` or `MERGE`), checking constraints and sequences.
6. Validate counts and application behaviour. Document the incident. Delete the temporary cluster.

```
pg_dump "host=<restored-endpoint> dbname=appdb user=dba_admin sslmode=require" \
  --table=public.orders --data-only --format=custom -f orders_recovery.dump
pg_restore --host=<prod-endpoint> --dbname=appdb --username=dba_admin --data-only \
  --table=orders --jobs=2 orders_recovery.dump         # restore into an empty or staging table
```

11.8 Other backup options

- **Cross-Region / cross-account:** copy snapshots (`aws rds copy-db-cluster-snapshot --source-region ... --kms-key-id ...`) or use AWS Backup. Share snapshots with `modify-db-cluster-snapshot-attribute`.
- **Export to Amazon S3:** `aws rds start-export-task` exports snapshot data in Parquet for analytics or archive.
- **Logical backups:** `pg_dump` and `pg_dumpall --globals-only` (roles are not in a database dump) are useful for portability and for small databases. Run them against a reader or a clone to avoid load on the writer.
- **Global Database:** physical cross-Region replication for low RPO disaster recovery (Chapter 12).

11.9 Backup verification schedule

Frequency	Action	Evidence to keep
Daily	Check `LatestRestorableTime` and snapshots	Alarm / dashboard screenshot
Monthly	Restore latest snapshot to a temporary cluster, run integrity checks and a sample application test	Restore time (RTO), checks passed
Quarterly	PITR to a chosen timestamp and recover a single table	Runbook timing and gaps found
Annually	Full DR exercise including cross-Region	Signed-off report

12. High Availability, Failover and Scaling

12.1 How failover works

- If the writer fails, Aurora promotes a reader based on **promotion tier** (0 is highest priority, 15 lowest; ties go to the largest instance).
- The cluster endpoint DNS record is updated to the new writer. Typical failover completes in about a minute or less; applications must reconnect.
- If the cluster has no readers, Aurora creates a new writer, which takes longer. **Always run at least one reader in another AZ for production.**
- Clients that cache DNS too long or hold dead TCP connections see longer outages. Set a low DNS TTL in the JVM/OS, enable TCP keepalives and use a driver with failover awareness (for example the AWS Advanced JDBC Wrapper) or RDS Proxy.

12.2 Planned failover (test or maintenance)

1. Announce the window; confirm the target reader is healthy and has low replica lag.
2. Optionally choose the target: `--target-db-instance-identifier`.
3. Run the failover, then watch events and application error rate.
4. Verify the new writer and that the old writer rejoined as a reader.

```
aws rds failover-db-cluster --db-cluster-identifier prod-apg --target-db-instance-identifier
    prod-apg-reader1
aws rds describe-events --source-type db-cluster --source-identifier prod-apg --duration 60
aws rds describe-db-clusters --db-cluster-identifier prod-apg \
  --query "DBClusters[0].DBClusterMembers[].[DBInstanceIdentifier,IsClusterWriter,PromotionTier]"
      --output table
```

12.3 Add, remove and tune readers

```
aws rds create-db-instance --db-instance-identifier prod-apg-reader2 --db-cluster-identifier
    prod-apg \
  --engine aurora-postgresql --db-instance-class db.r6g.xlarge --promotion-tier 2
      --availability-zone us-east-1c
aws rds delete-db-instance --db-instance-identifier prod-apg-reader2          # removes the reader
    only
aws rds modify-db-instance --db-instance-identifier prod-apg-reader1 --promotion-tier 1
    --apply-immediately

# Custom endpoint for analytics readers
aws rds create-db-cluster-endpoint --db-cluster-identifier prod-apg --db-cluster-endpoint-identifier
    analytics \
  --endpoint-type reader --static-members prod-apg-reader2
```

12.4 Vertical scaling (change instance class)

1. Take a manual snapshot (Chapter 11). Schedule a low-traffic window.
2. Modify each **reader** first, one at a time. Each modification restarts that instance.
3. Fail over to a resized reader so it becomes the writer.
4. Modify the former writer (now a reader) to the same class.
5. Verify parameters (memory-based defaults change with instance size) and review `max_connections`.

```
aws rds modify-db-instance --db-instance-identifier prod-apg-reader1 --db-instance-class
    db.r6g.2xlarge --apply-immediately
aws rds wait db-instance-available --db-instance-identifier prod-apg-reader1
aws rds failover-db-cluster --db-cluster-identifier prod-apg --target-db-instance-identifier
    prod-apg-reader1
aws rds modify-db-instance --db-instance-identifier prod-apg-writer --db-instance-class
```

```
    db.r6g.2xlarge --apply-immediately
```

12.5 Reader Auto Scaling

```
aws application-autoscaling register-scalable-target --service-namespace rds \
  --scalable-dimension rds:cluster:ReadReplicaCount --resource-id cluster:prod-apg \
  --min-capacity 1 --max-capacity 5

aws application-autoscaling put-scaling-policy --service-namespace rds \
  --scalable-dimension rds:cluster:ReadReplicaCount --resource-id cluster:prod-apg \
  --policy-name cpu-target-60 --policy-type TargetTrackingScaling \
  --target-tracking-scaling-policy-configuration '{
    "TargetValue": 60.0,
    "PredefinedMetricSpecification": {"PredefinedMetricType": "RDSReaderAverageCPUUtilization"},
    "ScaleInCooldown": 300, "ScaleOutCooldown": 120}'
```

12.6 Cluster cache management

Set `apg_ccm_enabled = 1` in the cluster parameter group and keep one reader at **promotion tier 0** with the same instance class as the writer. That reader continuously receives the list of hot buffers from the writer, so after failover it has a warm cache.

12.7 Disaster recovery: Aurora Global Database

- A Global Database has one primary Region and up to several read-only secondary Regions with storage-based replication (typically under a second of lag).
- **Switchover** (planned, no data loss) and **failover** (unplanned, possible small data loss) are supported through the console and CLI.
- Plan RPO and RTO, test with `aws rds switchover-global-cluster` in a drill, and keep application connection configuration Region-aware.
- Monitor `AuroraGlobalDBReplicationLag` and `AuroraGlobalDBRPOLag`.

```
aws rds create-global-cluster --global-cluster-identifier prod-apg-global
    --source-db-cluster-identifier <primary_cluster_arn>
aws rds switchover-global-cluster --global-cluster-identifier prod-apg-global
    --target-db-cluster-identifier <secondary_cluster_arn>
```

13. Upgrades and Maintenance

13.1 Maintenance windows and pending actions

```
aws rds modify-db-cluster --db-cluster-identifier prod-apg --preferred-maintenance-window
    sun:22:00-sun:22:30 \
  --preferred-backup-window 21:00-21:30
aws rds describe-pending-maintenance-actions --filters Name=db-cluster-id,Values=prod-apg
aws rds apply-pending-maintenance-action --resource-identifier <cluster_arn> \
  --apply-action system-update --opt-in-type immediate
```

13.2 Minor version upgrade (procedure)

1. Read the release notes for fixes, deprecations and extension updates.
2. Upgrade a **clone or staging cluster** first and run the application test suite.
3. Take a manual snapshot of production.
4. Upgrade the cluster in a low-traffic window. Aurora upgrades readers and writer in a coordinated way; expect a short connection interruption (some versions support zero-downtime patching when conditions allow).
5. Update extensions and verify.

```
aws rds modify-db-cluster --db-cluster-identifier prod-apg --engine-version <NEW_MINOR_VERSION>
    --apply-immediately
aws rds describe-db-clusters --db-cluster-identifier prod-apg --query
    "DBClusters[0].[Status,EngineVersion]"
```

13.3 Major version upgrade (procedure)

A major upgrade (for example 15 to 17) changes the on-disk catalog format and planner behaviour, and needs preparation.

1. **Check upgrade path:** `describe-db-engine-versions ... ValidUpgradeTarget` (Section 5.3).
2. **Read the release notes** and the AWS documentation for major version upgrades: removed features, changed defaults, extension compatibility.
3. **Run pre-checks** below and fix blockers.
4. **Rehearse on a clone:** upgrade the clone, run the application test suite, compare query plans and timings.
5. **Create new parameter groups** for the target family and copy your custom settings (families are not interchangeable).
6. **Take a manual snapshot**; notify stakeholders; schedule downtime (major upgrades involve an outage of the cluster).
7. **Run the upgrade** with `--allow-major-version-upgrade` and the new parameter groups.
8. **Post-upgrade:** update extensions, run `ANALYZE` (use `vacuumdb --analyze-in-stages`), re-check plans for top SQL, review logs and metrics.
9. **Keep the snapshot** until the application has run through a full business cycle.

```
-- Pre-checks (psql, in every database)
SELECT * FROM pg_prepared_xacts;                                    -- must be empty: commit or roll
    back
SELECT slot_name, plugin, slot_type, active FROM pg_replication_slots;   -- drop logical slots
    before upgrade
SELECT name, default_version, installed_version FROM pg_available_extensions
WHERE installed_version IS NOT NULL AND installed_version <> default_version;   -- extensions
    needing update
-- Also verify the "unsupported reg* data types" check from the AWS major-version-upgrade
    documentation
-- (columns using types such as regproc, regoper, regconfig, regnamespace in user tables block the
    upgrade).
```

```
-- Upgrade
aws rds modify-db-cluster --db-cluster-identifier prod-apg --engine-version <TARGET_VERSION> \
  --allow-major-version-upgrade --db-cluster-parameter-group-name prod-apg18-cluster \
  --db-instance-parameter-group-name prod-apg18-instance --apply-immediately

-- Post-upgrade (psql)
ALTER EXTENSION pg_stat_statements UPDATE;     -- repeat for each installed extension
-- shell:  vacuumdb --host=<endpoint> --username=<user> --all --analyze-in-stages --jobs=4
```

13.4 Lower-downtime alternatives

- **Blue/Green Deployments:** AWS creates a synchronized green copy of your cluster, you upgrade or change it, test, and then switch over typically within about a minute. Check the documentation for supported versions and limitations (logical replication prerequisites, DDL restrictions while the green environment is active).
- **Logical replication to a new cluster:** build a target cluster at the new version, replicate with a publication and subscription, and cut over when caught up. Sequences, large objects and DDL are not replicated automatically; plan for them.
- **AWS DMS:** useful for heterogeneous migrations and for some version moves.

13.5 Extension and certificate hygiene

- After every engine upgrade, run `ALTER EXTENSION <name> UPDATE` for installed extensions.
- Track the RDS CA certificate expiry for your clusters. When AWS announces a CA rotation, update the cluster CA and client trust bundles before the expiry date.
- Review deprecations of engine minor versions at least quarterly. Running on an unsupported minor version forces an automatic upgrade.

14. Security Administration

14.1 Role model (least privilege)

Create one **owner role** that owns objects, one **read/write role** and one **read-only role** per application. Applications log in with login roles that inherit from these group roles. Never use the master user in applications.

```
-- As master user / rds_superuser
CREATE ROLE app_owner NOLOGIN;
CREATE ROLE app_rw    NOLOGIN;
CREATE ROLE app_ro    NOLOGIN;

CREATE DATABASE appdb OWNER app_owner;
\c appdb
REVOKE ALL ON DATABASE appdb FROM PUBLIC;
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
CREATE SCHEMA app AUTHORIZATION app_owner;

GRANT CONNECT ON DATABASE appdb TO app_rw, app_ro;
GRANT USAGE ON SCHEMA app TO app_rw, app_ro;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA app TO app_rw;
GRANT SELECT ON ALL TABLES IN SCHEMA app TO app_ro;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA app TO app_rw;
ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA app
  GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO app_rw;
ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA app GRANT SELECT ON TABLES TO app_ro;
ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA app GRANT USAGE, SELECT ON SEQUENCES TO
    app_rw;

CREATE ROLE svc_orders LOGIN PASSWORD '<strong-password>' CONNECTION LIMIT 100 IN ROLE app_rw;
ALTER ROLE svc_orders SET statement_timeout = '60s';
ALTER ROLE svc_orders SET search_path = app, public;

-- Monitoring user (no data access)
CREATE ROLE mon_user LOGIN PASSWORD '<strong-password>' IN ROLE pg_monitor;
-- DBA users: grant rds_superuser to named individuals only
GRANT rds_superuser TO alice_dba;
```

14.2 Authentication

- Set `password_encryption = scram-sha-256` and reset old md5 passwords after changing it.
- Use **AWS Secrets Manager** for the master user (`--manage-master-user-password`) and rotate application secrets.
- Prefer **IAM database authentication** for services running on AWS (no long-lived DB passwords).
- Integrate human DBAs with SSO and named roles; avoid shared accounts.

IAM database authentication: step by step

1. Enable it on the cluster.
2. Create a database user and grant the `rds_iam` role.
3. Attach an IAM policy allowing `rds-db:connect` for that database user to the caller's IAM role.
4. Generate a token (valid 15 minutes) and use it as the password over SSL.

```
aws rds modify-db-cluster --db-cluster-identifier prod-apg --enable-iam-database-authentication
    --apply-immediately

-- psql
CREATE USER svc_report LOGIN;
GRANT rds_iam TO svc_report;
GRANT app_ro TO svc_report;

# IAM policy statement (resource uses the cluster resource ID, not the name)
{ "Effect": "Allow", "Action": "rds-db:connect",
  "Resource": "arn:aws:rds-db:us-east-1:<acct>:dbuser:cluster-ABCDEFGHIJKL/svc_report" }
```

```
export PGPASSWORD="$(aws rds generate-db-auth-token --hostname <endpoint> --port 5432 --username
    svc_report)"
psql "host=<endpoint> port=5432 dbname=appdb user=svc_report sslmode=verify-full
    sslrootcert=global-bundle.pem"
```

14.3 Network and encryption

- Place the cluster in **private subnets**; set `PubliclyAccessible=false`; allow 5432 only from application and bastion security groups.
- Require TLS: `rds.force_ssl = 1` and clients using `sslmode=verify-full`.
- **Encryption at rest** is chosen at creation. An unencrypted cluster cannot be encrypted in place. To encrypt: snapshot, copy the snapshot with a KMS key (encrypted), restore from the encrypted copy, cut over.
- Use a customer managed KMS key if you need key rotation control, key policies or cross-account sharing.
- Restrict who can call `rds:DeleteDBCluster`, `ModifyDBCluster`, `RestoreDBCluster*`, and enable **deletion protection**.

```
aws rds copy-db-cluster-snapshot --source-db-cluster-snapshot-identifier <unencrypted_snap> \
  --target-db-cluster-snapshot-identifier <encrypted_snap> --kms-key-id alias/prod-rds
aws rds restore-db-cluster-from-snapshot --db-cluster-identifier prod-apg-enc \
  --snapshot-identifier <encrypted_snap> --engine aurora-postgresql --kms-key-id alias/prod-rds
```

14.4 Auditing with pgaudit

1. Add `pgaudit` to `shared_preload_libraries` in the cluster parameter group and reboot.
2. Create the extension in each database to audit: `CREATE EXTENSION pgaudit;`.
3. Set `pgaudit.log` (for example `ddl, role`, or `write` for data changes). For object-level audit, create a role named by `pgaudit.role` and grant it privileges on the audited tables.
4. Ship logs to CloudWatch Logs; set retention and alarms on suspicious patterns (new superuser grants, privilege changes, DROP statements).

```
-- parameter group:  pgaudit.log = ddl,role     pgaudit.log_catalog = 0
CREATE EXTENSION pgaudit;
CREATE ROLE rds_pgaudit;                              -- set pgaudit.role = rds_pgaudit in the
    parameter group
GRANT SELECT ON app.customers TO rds_pgaudit;         -- object-level auditing: SELECTs on this
    table are logged
```

For continuous, tamper-resistant database activity auditing at scale consider **Database Activity Streams**.

14.5 Access review queries

```
-- Roles and attributes
SELECT rolname, rolsuper, rolcreaterole, rolcreatedb, rolcanlogin, rolconnlimit, rolvaliduntil
FROM pg_roles WHERE rolname NOT LIKE 'pg\_%' ORDER BY rolname;
-- Who is in rds_superuser and other powerful roles
SELECT r.rolname AS role, m.rolname AS member FROM pg_auth_members am
JOIN pg_roles r ON r.oid = am.roleid JOIN pg_roles m ON m.oid = am.member
WHERE r.rolname IN ('rds_superuser','rds_replication','rds_iam');
-- Table privileges for a role
SELECT grantee, table_schema, table_name, privilege_type FROM information_schema.role_table_grants
WHERE grantee = 'app_ro' ORDER BY 2,3;
```

14.6 Security checklist

Item	Expected state
Public accessibility	Disabled; only private subnets
TLS	`rds.force_ssl=1`; clients verify the certificate
Encryption at rest	Enabled with KMS key
Master credentials	Managed by Secrets Manager, not used by applications
Named admin accounts	Individuals only; `rds_superuser` membership reviewed monthly
Auditing	pgaudit and/or Database Activity Streams configured; logs retained
Deletion protection	Enabled on production clusters
Backups	Retention meets policy; copies in another Region/account for critical data
Patching	Running a supported minor version; CA certificate current

15. Replication, Logical Replication and Migration

15.1 Aurora replicas versus logical replication

Aurora replicas (readers) are physical and share the storage volume; no setup beyond adding an instance. **Logical replication** (publications and subscriptions) replicates selected tables row by row and can feed another Aurora cluster, an RDS or self-managed PostgreSQL, or CDC tools such as AWS DMS and Debezium. It is also the base for low-downtime migrations.

15.2 Set up logical replication (publisher in Aurora)

1. Set `rds.logical_replication = 1` in the **cluster** parameter group and reboot the instances. This sets `wal_level` to logical.
2. Verify with `SHOW wal_level;` (expect `logical`).
3. Grant `rds_replication` to the replication user.
4. Create a publication on the source and a subscription on the target.
5. Monitor slots and lag. **Drop slots that are no longer used.**

```
-- Source (publisher)
CREATE ROLE repl_user LOGIN PASSWORD '<strong-password>';
GRANT rds_replication TO repl_user;
GRANT SELECT ON ALL TABLES IN SCHEMA app TO repl_user;
CREATE PUBLICATION pub_app FOR TABLES IN SCHEMA app;        -- or FOR TABLE app.orders,
    app.customers

-- Target (subscriber): schema must already exist (pg_dump --schema-only)
CREATE SUBSCRIPTION sub_app
  CONNECTION 'host=<source-endpoint> port=5432 dbname=appdb user=repl_user password=<pw>
      sslmode=require'
  PUBLICATION pub_app;

-- Monitor
SELECT slot_name, active, pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn))
    AS lag
FROM pg_replication_slots WHERE slot_type = 'logical';
SELECT * FROM pg_stat_subscription;                          -- on subscriber
SELECT pg_drop_replication_slot('<slot_name>');              -- cleanup an abandoned slot
    (publisher)
```

Warning: An inactive or lagging logical replication slot forces the cluster to keep transaction logs, which grows `TransactionLogsDiskUsage` and can fill local storage. Alarm on `OldestReplicationSlotLag` and `TransactionLogsDiskUsage`.

15.3 Migrating a database to Aurora PostgreSQL (plan)

1. **Assess:** extensions, data types, size, change rate, downtime tolerance. Use AWS Schema Conversion Tool and DMS assessments for non-PostgreSQL sources.
2. **Build the target:** cluster, parameter groups, roles, extensions (Chapters 5, 6, 14).
3. **Load schema:** `pg_dump --schema-only` and `pg_restore` (pre-data, then post-data after loading).
4. **Load data:** `pg_dump -Fd -j 8` and `pg_restore -j 8` for small outages; logical replication or DMS CDC for near-zero downtime.
5. **Validate:** row counts, checksums on sample ranges, sequence values, application tests, performance baselines.
6. **Cut over:** stop writes, wait for lag zero, fix sequences (`setval`), repoint application, keep the source read-only as a fallback.
7. **After cutover:** `ANALYZE`, monitor top SQL, remove replication objects.

```
pg_dump -Fd -j 8 -h <source> -U <user> -d appdb -f /data/appdb_dump
pg_restore -j 8 -h <aurora-writer-endpoint> -U dba_master -d appdb --no-owner --no-privileges
    /data/appdb_dump
-- After load, align sequences with table maxima, then ANALYZE
SELECT setval(pg_get_serial_sequence('app.orders','id'), (SELECT coalesce(max(id),1) FROM
    app.orders));
```

16. Storage and Temporary Space

16.1 Two kinds of storage

Storage	What lives there	Metric	Behaviour
Cluster volume (shared)	Tables, indexes, WAL/transaction logs for recovery	`VolumeBytesUsed`	Grows automatically up to 128 TiB; space from dropped or truncated objects is reclaimed dynamically.
Local instance storage	Temp files for sorts/hashes, temp tables, DB logs, some caches	`FreeLocalStorage`	Fixed by instance class. If it runs out, queries fail with "No space left on device" and the instance can restart.

16.2 Investigate cluster volume growth

```
SELECT datname, pg_size_pretty(pg_database_size(datname)) FROM pg_database ORDER BY
    pg_database_size(datname) DESC;

SELECT n.nspname, c.relname, c.relkind, pg_size_pretty(pg_total_relation_size(c.oid)) AS total,
       pg_size_pretty(pg_relation_size(c.oid)) AS heap, pg_size_pretty(pg_indexes_size(c.oid)) AS
           indexes
FROM pg_class c JOIN pg_namespace n ON n.oid=c.relnamespace
WHERE c.relkind IN ('r','m') AND n.nspname NOT IN ('pg_catalog','information_schema')
ORDER BY pg_total_relation_size(c.oid) DESC LIMIT 20;
```

- Large and growing **bloat** (Chapter 8): vacuum, repack, reindex.
- Large **logs/audit tables** or queue tables that never get purged: add retention and partitioning.
- Inactive **logical replication slots** or long transactions holding old data: see Chapter 15 and Runbook R11.
- Remember that **snapshots and clones** have their own cost; delete unused ones.

16.3 Investigate local storage (temp) pressure

```
-- Temp files per database since stats reset
SELECT datname, temp_files, pg_size_pretty(temp_bytes) FROM pg_stat_database ORDER BY temp_bytes
    DESC;
-- Queries currently generating temp files: check Performance Insights, or the log with
    log_temp_files enabled
SELECT queryid, calls, temp_blks_written, left(query,100) FROM pg_stat_statements
ORDER BY temp_blks_written DESC LIMIT 10;
```

1. Find the top temp-spilling SQL (`pg_stat_statements.temp_blks_written`, `log_temp_files`).
2. Fix the query: add an index to avoid the sort, filter earlier, avoid `SELECT DISTINCT` over huge sets, replace giant CTE materialization.
3. If appropriate, raise `work_mem` for that role only, and weigh the memory risk.
4. Set `temp_file_limit` per role so one query cannot consume all local space.
5. Reduce `rds.log_retention_period` or log verbosity if logs are large.
6. If the workload legitimately needs more local space, scale to a larger instance class (or one with local NVMe/Optimized Reads).

17. Troubleshooting Runbooks

Each runbook has the same structure: **Symptoms**, **Diagnose**, **Fix**, **Prevent**. Start with the one that matches the alert.

R1. High CPU

Symptoms

`CPUUtilization` above 80%, rising latency, Performance Insights shows `CPU` wait dominating and DB load above vCPU count.

Diagnose

1. Performance Insights: slice by SQL. Is one statement or one application responsible?
2. `pg_stat_statements` by total time (Section 7.6) and by calls (an N+1 pattern shows as huge call counts).
3. Check for a sudden change: deployment, new report, stats stale after bulk load, failed autovacuum, plan regression.
4. Enhanced Monitoring: per-process CPU to tell backend queries from autovacuum or parallel workers.

Fix

- Cancel runaway queries (`pg_cancel_backend`). Add or fix indexes; run `ANALYZE` on tables with stale statistics.
- Move reporting to readers or a custom endpoint. Reduce parallel query settings if CPU is oversubscribed.
- Throttle or batch application jobs; fix N+1 query patterns.
- If load is legitimate and sustained: scale up the instance class or add readers.

Prevent

CPU alarm, weekly top-SQL review, plan management for critical queries.

R2. Too many connections / connection exhaustion

Symptoms

Errors `FATAL: too many connections` or `remaining connection slots are reserved`; `DatabaseConnections` near `max_connections`; memory pressure.

Diagnose

```
SELECT usename, application_name, client_addr, state, count(*) FROM pg_stat_activity
GROUP BY 1,2,3,4 ORDER BY 5 DESC LIMIT 20;
SELECT count(*) FILTER (WHERE state='idle') idle, count(*) FILTER (WHERE state='idle in
    transaction') idle_in_tx,
       count(*) FILTER (WHERE state='active') active, count(*) total FROM pg_stat_activity;
```

Fix

- Terminate abandoned idle sessions: `SELECT pg_terminate_backend(pid) FROM pg_stat_activity WHERE state='idle' AND now()-state_change > interval '30 minutes' AND usename = '<app_user>';`
- Reduce application pool sizes; introduce RDS Proxy or PgBouncer; fix connection leaks (clients that open a connection per request).
- Use role-level `CONNECTION LIMIT` to stop one application from starving others.
- Scaling the instance class raises the default `max_connections`, but pooling is the durable fix.

R3. Replica lag and reader query cancellations

Symptoms

`AuroraReplicaLag` high; stale reads; errors such as `canceling statement due to conflict with recovery` on readers.

Diagnose

1. On the writer: `SELECT server_id, replica_lag_in_msec FROM aurora_replica_status();` Is one reader or all readers lagging?
2. Check writer load: heavy write bursts (bulk loads, large updates, vacuum, index builds) increase redo for readers to apply.
3. Check the lagging reader: CPU saturation or long-running queries holding old snapshots.
4. Check instance class: readers smaller than the writer lag under heavy write load.

Fix

- Break bulk writes into smaller batches; run big maintenance off-peak.
- Resize the lagging reader to match the writer; add readers to spread read load.
- Cancel long reader queries or move them to a dedicated reader; tune `max_standby_streaming_delay` / related reader conflict parameters per AWS guidance if conflicts are the issue.
- Applications needing read-your-writes should read from the writer.

R4. Slow query or sudden plan regression

1. Identify the statement (`pg_stat_statements`, Performance Insights). Capture parameters of a slow execution.
2. Run `EXPLAIN (ANALYZE, BUFFERS)` and compare with the earlier plan if saved (plan management, logs with `auto_explain`).
3. Check statistics freshness: `last_analyze`, `last_autoanalyze`; run `ANALYZE`.
4. Check bloat and visibility map; vacuum if needed.
5. Try fixes in a clone: new index, rewrite, `default_statistics_target`, extended statistics.
6. As a stop-gap, pin the plan with `apg_plan_mgmt` or `pg_hint_plan`; open a ticket for the permanent fix.

R5. Idle-in-transaction and long-running transactions

They hold locks, block DDL, prevent vacuum from removing dead rows and increase the transaction ID age.

1. Find them with the long-transaction query (Section 7.5) and check `backend_xmin` age (Section 8.2).
2. Identify the application and ask for a code fix (commit/rollback in `finally`, no user wait inside a transaction).
3. Terminate when required: `SELECT pg_terminate_backend(<pid>);`
4. Prevent: `idle_in_transaction_session_timeout` per role; `statement_timeout` for batch users; alerts for transactions older than 15 minutes.

R6. Lock pile-up (application hangs)

1. Run the blocking query (Section 10.1); find the root blocker and what it holds.
2. If the blocker is a DDL or long transaction waiting itself behind another session, resolve the oldest in the chain.
3. Cancel/terminate the root blocker (Section 10.2). Confirm the queue drains.
4. Review how the blocker started: DDL without `lock_timeout`, forgotten session, batch job without limit.

R7. Low local storage or disk full

1. Check `FreeLocalStorage` trend and `VolumeBytesUsed`. Determine whether it is local (temp/logs) or the cluster volume.
2. Local: apply Section 16.3 (find temp-heavy SQL, cancel it, limit with `temp_file_limit`, shorten log retention).
3. Volume: find largest relations, bloat, logical slots, old data (Section 16.2).
4. Check `TransactionLogsDiskUsage` and replication slots (R11).
5. If urgent and the instance class is too small, scale up or fail over to a reader with more free local storage.

R8. Transaction ID wraparound risk

Symptoms

`MaximumUsedTransactionIDs` above 1 billion and rising; log messages about wraparound; autovacuum running constantly with "to prevent wraparound".

Fix (in order)

1. Find the oldest database and tables (Section 8.1). Find what holds back the horizon (Section 8.2): old transactions, prepared transactions, replication slots.
2. Terminate old transactions; roll back orphan prepared transactions (`ROLLBACK PREPARED '<gid>'`); drop unused replication slots.
3. Raise autovacuum capacity for the incident: increase `autovacuum_vacuum_cost_limit`, maintenance memory; ensure `rds.adaptive_autovacuum = 1`.
4. Run manual freeze on the oldest tables, biggest age first: `VACUUM (FREEZE, VERBOSE) <table>;` In a session with large `maintenance_work_mem`.
5. Re-check `age(datfrozenxid)` until it drops well below 200-500 million.
6. If the database already refuses writes, follow AWS guidance for the wraparound-protection state and open an AWS Support case at once.

Prevent

Alarm at 500 million, tune per-table autovacuum for large tables, avoid long transactions, drop stale slots.

R9. Unexpected restart or failover

1. `aws rds describe-events` for the instance and cluster in the incident window: failover, reboot, "out of memory", storage events.
2. CloudWatch: `FreeableMemory`, `SwapUsage`, CPU, connections around the event time. A sharp drop in freeable memory before restart suggests memory exhaustion.
3. PostgreSQL log around the time: look for `terminated by signal 9`, `out of memory`, `PANIC`, `checkpoint`/`crash recovery`.
4. Memory causes: too many connections, large `work_mem` multiplied by parallel sessions, huge `maintenance_work_mem`/`autovacuum_work_mem`, many cached plans. Reduce and re-test.
5. Confirm applications reconnected correctly; fix DNS caching or retry logic if recovery was slow.
6. If cause is unclear or repeated, open an AWS Support case with timestamps, instance ID and the log excerpts.

R10. Cannot connect to the cluster

Follow the ordered checks in Section 4.4: DNS, network, security group, route/NACL, authentication, SSL, connection limit, instance state.

R11. Logical replication slot lag or log growth

1. Check slots: `SELECT slot_name, active, restart_lsn, confirmed_flush_lsn FROM pg_replication_slots;` and CloudWatch `OldestReplicationSlotLag`, `TransactionLogsDiskUsage`.
2. If the consumer is down (DMS task stopped, subscriber offline): restart the consumer, or if it is abandoned drop the slot with `pg_drop_replication_slot`.
3. If the consumer is slow: scale the consumer, tune its batch settings, reduce publication scope.
4. Check for very large transactions that the consumer must decode.

R12. High memory use or swapping

1. Enhanced Monitoring: which processes use memory? Many backend processes = connection issue (R2).
2. Look for queries with big sorts/hashes across many parallel workers; check `work_mem`, `hash_mem_multiplier`, `max_parallel_workers_per_gather`.
3. Lower per-session memory settings; set role-level limits; add pooling.
4. Scale up the instance class if the working set genuinely does not fit.

18. Extensions and Useful Tools

Extension / tool	Purpose	Notes
`pg_stat_statements`	SQL execution statistics	Preload library; essential.
`pgaudit`	Audit logging	Preload library; see Section 14.4.
`pg_cron`	Scheduled jobs inside PostgreSQL	Preload library; set `cron.database_name`.
`pg_repack`	Online table/index rebuild	Needs PK/unique index and free space.
`pgstattuple`	Bloat measurement	Use `pgstattuple_approx` on big tables.
`pg_partman`	Partition automation	Use with `pg_cron` for maintenance.
`pg_hint_plan`	Query hints	Preload library; last-resort tuning.
`apg_plan_mgmt`	Query plan management	Aurora-specific; prevents regressions.
`auto_explain`	Log plans of slow queries	Preload library; set threshold carefully.
`pg_trgm`, `btree_gin`, `btree_gist`	Text search and index types	Common application needs.
`postgis`	Geospatial	Update after PostGIS version changes.
`pgvector`	Vector similarity search	Memory-sizing and index choice matter (HNSW/IVFFlat).
`aws_s3`, `aws_lambda`	Import/export with S3, invoke Lambda	Requires IAM roles associated with the cluster.

18.1 Installing an extension

```
-- Which extensions are available and installed?
SELECT name, default_version, installed_version FROM pg_available_extensions ORDER BY name;
-- Install in the database (requires rds_superuser or suitable privilege)
CREATE EXTENSION IF NOT EXISTS pg_trgm;
-- Libraries needing shared_preload_libraries: edit the cluster parameter group, reboot, then CREATE
    EXTENSION.
```

18.2 Scheduling jobs with pg_cron

```
-- parameter group: shared_preload_libraries = pg_stat_statements,pg_cron ; cron.database_name =
    appdb  (reboot)
CREATE EXTENSION pg_cron;
SELECT cron.schedule('nightly-analyze', '0 2 * * *', 'ANALYZE app.orders');
SELECT cron.schedule('purge-old-events', '30 3 * * *', $$DELETE FROM app.events WHERE created_at <
    now() - interval '90 days'$$);
SELECT * FROM cron.job;                      -- list jobs
SELECT * FROM cron.job_run_details ORDER BY start_time DESC LIMIT 10;
SELECT cron.unschedule('nightly-analyze');
```

18.3 Declarative partitioning basics

```
CREATE TABLE app.events (id bigint GENERATED ALWAYS AS IDENTITY, created_at timestamptz NOT NULL,
    payload jsonb,
                         PRIMARY KEY (id, created_at)) PARTITION BY RANGE (created_at);
CREATE TABLE app.events_2026_10 PARTITION OF app.events FOR VALUES FROM ('2026-10-01') TO
    ('2026-11-01');
-- Retention: detach and drop old partitions instead of DELETE (fast and no bloat)
ALTER TABLE app.events DETACH PARTITION app.events_2026_07 CONCURRENTLY;
DROP TABLE app.events_2026_07;
```

18.4 Import/export with Amazon S3

```
-- cluster must have an IAM role associated for the s3Import / s3Export feature
-- aws rds add-role-to-db-cluster --db-cluster-identifier prod-apg --feature-name s3Import
    --role-arn <role_arn>
CREATE EXTENSION aws_s3 CASCADE;
SELECT aws_s3.table_import_from_s3('app.stage_orders', '', '(format csv, header true)',
       aws_commons.create_s3_uri('my-bucket', 'orders/orders.csv', 'us-east-1'));
```

19. Cost, Change Management and Operations Governance

19.1 Cost controls

- **Right-size** instances using CPU, memory and cache-hit trends; Graviton classes often reduce cost.
- **Aurora I/O-Optimized vs Standard:** I/O-Optimized has higher instance/storage cost but no per-request I/O charge; it pays off when I/O charges are a large share (commonly above about 25% of the Aurora bill). Compare using Cost Explorer before switching.
- **Serverless v2** for dev/test and variable workloads; set a sensible minimum ACU.
- **Stop** non-production clusters when idle (`aws rds stop-db-cluster`; a stopped cluster restarts automatically after 7 days).
- Delete old manual snapshots and unused clones; set lifecycle in AWS Backup.
- Tag everything (`env`, `owner`, `cost-center`) and review cost per cluster monthly.
- Keep Performance Insights retention at the free tier unless you need longer history.

19.2 Change management checklist (any production change)

1. Describe the change, risk, rollback plan and expected duration. Get approval.
2. Rehearse on a clone with realistic data. Record timings.
3. Take a manual snapshot (Procedure A) and record the snapshot name.
4. Announce the window; confirm monitoring and on-call.
5. Apply the change; set `lock_timeout`/`statement_timeout` for DDL; watch metrics.
6. Validate: functional checks, top SQL, error rate, replica lag.
7. Close out: update documentation, delete temporary resources, record lessons.

19.3 Incident record template

Field	Content
Summary / impact	What users saw; start/end time (UTC); services affected
Detection	Alarm, user report, or monitoring; time to detect
Timeline	Key actions with timestamps
Root cause	Technical cause and contributing factors
Resolution	Exact commands or changes that fixed it
Prevention	Alarms, parameter changes, code fixes, runbook updates with owners and due dates

Appendix A. AWS CLI Quick Reference

Task	Command
List clusters	`aws rds describe-db-clusters`
List instances of a cluster	`aws rds describe-db-instances --filters Name=db-cluster-id,Values=<cluster>`
Create cluster / instance	`aws rds create-db-cluster` / `create-db-instance`
Modify cluster settings	`aws rds modify-db-cluster --db-cluster-identifier <c> --apply-immediately ...`
Modify instance	`aws rds modify-db-instance --db-instance-identifier <i> ...`
Failover	`aws rds failover-db-cluster --db-cluster-identifier <c> [--target-db-instance-identifier <i>]`
Reboot instance	`aws rds reboot-db-instance --db-instance-identifier <i>`
Stop / start cluster	`aws rds stop-db-cluster` / `start-db-cluster`
Snapshot	`aws rds create-db-cluster-snapshot`
Restore from snapshot	`aws rds restore-db-cluster-from-snapshot`
PITR / clone	`aws rds restore-db-cluster-to-point-in-time` (`--restore-type copy-on-write` for a clone)
Parameter groups	`create-db-cluster-parameter-group`, `modify-db-cluster-parameter-group`, `describe-db-cluster-parameters`
Events and logs	`describe-events`, `describe-db-log-files`, `download-db-log-file-portion`
Pending maintenance	`describe-pending-maintenance-actions`, `apply-pending-maintenance-action`
Delete (careful)	`aws rds delete-db-instance`, `delete-db-cluster --final-db-snapshot-identifier <name>` (disable deletion protection first)

Appendix B. SQL Quick Reference

```
-- Version and instance
SELECT version(), aurora_version(), aurora_db_instance_identifier(), pg_is_in_recovery();

-- Non-default settings
SELECT name, setting, unit, source FROM pg_settings WHERE source NOT IN ('default','override') ORDER
    BY name;

-- Pending restart
SELECT name, setting, pending_restart FROM pg_settings WHERE pending_restart;

-- Largest tables and indexes
SELECT relname, pg_size_pretty(pg_total_relation_size(relid)) FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC LIMIT 10;

-- Session control
SELECT pg_cancel_backend(<pid>);       -- cancel the current query
SELECT pg_terminate_backend(<pid>);    -- end the session

-- Stats reset time and uptime
SELECT stats_reset FROM pg_stat_database WHERE datname = current_database();
SELECT now() - pg_postmaster_start_time() AS uptime;

-- Sequences close to exhaustion (integer columns): check max value of identity/serial columns
    regularly
SELECT schemaname, sequencename, last_value, max_value,
       round(100.0*last_value/max_value,2) AS pct_used FROM pg_sequences WHERE last_value IS NOT
           NULL
ORDER BY pct_used DESC LIMIT 10;

-- Role-level or database-level setting overrides
SELECT coalesce(r.rolname,'(all roles)') role, coalesce(d.datname,'(all dbs)') db, s.setconfig
FROM pg_db_role_setting s LEFT JOIN pg_roles r ON r.oid=s.setrole LEFT JOIN pg_database d ON
    d.oid=s.setdatabase;
```

Appendix C. References

Primary sources. Always confirm version-specific details here before production changes.

- Amazon Aurora User Guide: https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/
- Best practices with Amazon Aurora PostgreSQL (User Guide, topic `AuroraPostgreSQL.BestPractices`)
- Common DBA tasks for Aurora PostgreSQL (User Guide, topic `Appendix.PostgreSQL.CommonDBATasks`)
- Aurora PostgreSQL query plan management (User Guide, topic `AuroraPostgreSQL.Optimize`)
- Amazon Aurora PostgreSQL release notes and extension versions: https://docs.aws.amazon.com/AmazonRDS/latest/AuroraPostgreSQLReleaseNotes/
- AWS Database Blog: https://aws.amazon.com/blogs/database/
- PostgreSQL documentation: https://www.postgresql.org/docs/

Document status: this is a general-purpose handbook. Replace example identifiers, versions and thresholds with your own standards, and review it with your security and operations teams before adopting it as a formal runbook.